我们如何让 AI 自己找到真实漏洞——一套渗透测试 Agent 的认知架构

不是让 AI 调用扫描器，是让 AI 像安全研究员一样思考

你可能听说过用 AI 做渗透测试：给模型一堆工具，让它自己调。听起来很酷，但实际跑起来，它会自信地告诉你"发现 SQL 注入"——你一看证据，是页面返回了 200。然后你问它下一步怎么验证，它说"可能存在注入"。来回十轮，还是"可能"。问题不在工具，在脑子。AI 不缺发请求的能力，缺的是：知道什么值得测、知道什么时候该换方向、知道"可能"不算漏洞。我们做了一套认知架构来解决这个问题。这篇文章讲的不是 API 怎么用，是设计背后的每一个决策，以及为什么这么做。

一、Agent ≠ 让 AI 调工具

先说清楚一个概念。很多人以为 AI Agent 就是"给 LLM 一堆工具，让它自己调"。这叫自动化，不叫智能。打个比方：你给一个实习生一台电脑和一堆扫描器，说"去挖漏洞"。如果他只是机械地跑 nmap、跑 nuclei、跑 sqlmap，那他只是在执行脚本。但如果他先看目标是什么技术栈，根据技术栈推断最可能的漏洞类型，针对性地设计测试方案，测完发现方向不对就换思路——这才是智能。Agent 的核心公式是：

> Agent 能力 = 循环次数 × 工具质量 × 规划能力

「循环次数」是它能跑多少轮，「工具质量」是它手里的扫描器好不好用，但决定性的因素是「规划能力」——第 10 轮做的决策，是否比第 1 轮更聪明。

二、OODA 循环：比 ReAct 多一步「想清楚再动手」

业界最常见的 Agent 循环叫 ReAct：想一步，做一步，看结果，再想。听起来合理，但有个问题："想"和"做"在同一个 LLM 调用里完成。这就像让一个人同时当将军和当士兵——将军负责想战略，士兵负责执行。如果你让同一个人又想又做，他容易把"我应该先侦察"和"我直接开枪"混在一起。我们用的是 OODA 循环，多了一步 Orient（定位）：

```
观察（Observe）→ 定位（Orient）→ 决策（Decide）→ 执行（Act）→ 回到观察...
```

具体来说：

- **Observe** ：工具返回的结果，更新到信念网络里
- **Orient** ：从原始结果中提取信号（这个响应里有没有 SQL 错误？有没有 WAF 拦截？有没有新发现的端点？）——这一层是**确定性代码**，不靠 LLM
- **Decide** ：规划层根据当前信念状态决定下一步（不是随机选，是有数学排序的）
- **Act** ：执行工具，结果落盘，在线学习记录

关键设计：规划层是三层降级的。第一层用树搜索（像下棋一样模拟未来几步），如果失败就降级到 LLM 策略师（让模型出主意），再失败就降级到纯规则引擎（用数学公式排序，零 LLM 依赖）。任何一层失败都不让整个系统停摆——这是工程上最重要的设计。

> 小白版：就像下棋，先想远一点，想不动了就只想下一步，实在不想了就按规矩走。不会因为脑子转不动就停在那。

三、信念网络：用数学让"没证据的 critical"排不上号

传统扫描器的记忆是扁平的——"试过 X，返回 200"。我们维护的是一张概率图。简单说：Agent 的脑子里有一个"待办清单"，每条待办是一个假设，比如"登录接口可能有 SQL 注入"。每个假设有四个关键属性：

- **置信度** （0-1）：我有多相信这个假设？新证据会贝叶斯更新它
- **影响力** ：如果这是真的，严重程度多高？
- **证据** ：支撑和反驳它的证据列表
- **状态** ：pending（待验证）/ confirmed（已确认）/ rejected（已否决）/ phenomenon（暂存）

假设的排序公式是整个系统的反幻觉核心：

```
优先级 = 置信度 × 影响力权重 × 新颖度 × 证据惩罚 + 链式奖励
```

其中证据惩罚 = 0.25是关键。LLM 最爱报 critical 级别的漏洞——"可能存在远程代码执行"，听起来很厉害，但没有任何实证。在我们的公式里，一个没证据的 critical 假设（0.5 × 1.0 × 0.25 = 0.125），排在一个有证据的 medium 假设（0.5 × 0.4 × 1.0 = 0.20）后面。

> 小白版：就像公司评绩效，你说"我可能做了个大项目"只能得 0.125 分，别人说"我做了个小项目，这是交付物"能得 0.20 分。不是看你说得多厉害，是看你拿得出多少证据。

> 内行版：endpoint_ref 绑定让假设指向真实接口（不是模型凭空想象的 URL），evidence_penalty 对未绑定 + 未证据化的假设施加 75% 折扣。两者叠加，LLM 幻觉的 critical 在 EIG 排序中自然沉底，不需要人工干预。

四、黑板架构：TypeScript 类型系统让 Worker 写不了数据

多 Agent 协作最常见的 bug 是竞态——两个 Worker 同时写同一块数据，结果不一致。传统做法是加锁，但锁是 runtime 约定，靠程序员遵守。我们的做法：用类型系统在编译期就让 Worker 写不了。Worker 拿到的接口叫 ReadonlyBlackboard，只有两个方法：query（查数据）和 subscribe（订阅更新）。根本没有publish方法。不是靠注释说"你别写"，是编译器直接报错——你调 board.publish(...) 就编译不过。

```
黑板（唯一记忆存储）
 ↑ 只有它能写
调度器（Dispatcher）
 ↑ 只读
执行者（Worker 1）（Worker 2）（Worker 3）...
```

> 小白版：就像公司只有一个会计能动账本，其他员工只能看账本。不是靠规章制度约束，是会计室的门只有会计有钥匙。
>
> 内行版：run(intent: Intent, board: ReadonlyBlackboard): Promise<Fact[]>——Worker 的返回类型是 Fact[]，它把发现交给 Dispatcher，由 Dispatcher 统一 publish。并发路径下 reap() 是单线程的，10 个 Worker 并发写入点仍然只有一个。这不是锁，是架构约束。

五、Intent 的两种死法：crash 和 cancel 为什么要分开

每个"探索方向"在系统里叫一个 Intent，有状态机：open → claimed → concluded。但"Worker 中途挂了"和"监督者判断这方向没价值"是两种完全不同的失败：

- **crash** （Worker 非自愿死亡）→ 租约超时后回收到 open 池 → 其他 Worker 可以重新认领。**值得重试。**
- **cancel** （监督者主动叫停）→ 进入 cancelled 终态 → 永不复活。**不值得浪费资源。**

> 小白版：就像外卖骑手。骑手路上出了车祸（crash），订单会重新派给别的骑手。但如果商家告诉你这道菜卖完了（cancel），你不会再等，直接退款。
>
> 内行版：chain_id + step 机制支持跨 Worker 的多步攻击链——上一步的 Fact 成为下一步的起点，nextStep(chainId, direction) 自动递增 step。这支持"先发现入口 → 确认无校验 → 注入 PoC"这种长链构造。

六、Guardian：7 道关卡，让假漏洞过不去

这是对抗 LLM 幻觉最重的一道防线。纯代码，零 API 成本，跑在每个即将确认的漏洞上： 第 1 关：垃圾关键词（27 个）——"加固评分"、"SSL 证书过期"、"CORS 配置缺失"这种扫描器噪音，不算漏洞，降级。 第 2 关：投机措辞——"可能"、"疑似"、"might"、"possibly"。说了等于没说，降级。 第 3 关：条件句投机——"如果 X 就会 Y"，但 X 从没发生过。降级。 第 4 关：描述太短——不到 30 字的漏洞描述，无法评估，直接拒绝。 第 5 关：没有证据——一个字的证据都没有，降级。 第 6 关：没有"实证谓词"——这是最核心的一关。正则检查：描述或证据里有没有"我做了 X，观察到 Y"这种实证？比如 HTTP 状态码、响应内容、注入结果、文件写入确认。Web 证据和客户端证据都覆盖（ipcMain、nodeIntegration、AKIA 密钥、CORS 头、127.0.0.1:port 都算实证词）。 第 7 关：全通过→ 交 LLM 做二次验证（判断是否是误报、调整严重度）。

最关键的设计：降级不丢弃。被降级的发现变成 phenomenon（现象），留在系统里，等后续有新证据了可以拉回来重新验证。这防止了"误杀"——一个暂时没证据但方向正确的发现，不会被永久扔掉。

> 小白版：就像法院判案。证据不足不能定罪，但不代表无罪——可以"存疑待查"，等新证据来了再审。
>
> 内行版：demoteToPhenomenon(id, reason) 设置 status = "phenomenon" + demote_reason。getPendingHypotheses() 过滤掉 phenomenon，但 promoteFromPhenomenon(id) 可以把它拉回 pending。Reason 的 prompt 里能看到 phenomena 列表和降级原因，所以它知道哪些方向需要补充证据。

七、元认知：在 Agent 钻牛角尖时把它拽出来

Reason（规划者）的问题是会陷入局部最优——连续 3 轮都在测同一个接口的 SQL 注入，死活不换方向。这时候需要一个"旁观者"——Metacog（元认知）。它的作用不是"做得更好"，而是"质疑当前方向"。触发条件很有意思——不是定时触发，是恰恰在 Reason 沉默的时候触发：

- 连续 3 次没新发现 → 可能方向错了
- 连续 2 次没新发现 + 所有待办都是低价值 → 在产出噪音
- 人工打了 metacog hint → 强制发散
- 每 3 个 productive turn → 周期性强制发散

Metacog 用 5 个创造性框架生成假设：

> 小白版：就像下棋时你在想"怎么吃掉对方的车"，旁观者说"别想了，你应该看看对方的马能不能将死你"。你陷进去的时候，需要有人把你拽出来。
>
> 内行版：Metacog 的每个假设强制包含四要素契约：动词（mutate/inject/replay）+ 具体对象（参数名/方法名/路径）+ 落盘路径（证据存哪）+ 命中信号（什么响应算成功）。Reason 产

> 出是方向性的"去测试这个接口"，Metacog 产出是可执行的"把 ipcRenderer.send('pushUpdate', {productName: 'app"; touch /tmp/PWNED; #'}) 注入到 session_id=X 的渲染进程，如果 /tmp/PWNED 存在则命中"。

八、同一台机器，两套打法：Web 黑盒 vs 客户端灰盒

渗透测试有两个完全不同的战场：

- 黑盒 Web 渗透就像在黑暗中找门——你不知道目标有多少个入口，核心能力是"发现下一个入口"。信息是渐进获取的：先扫端口，再看服务，再找漏洞。
- 灰盒客户端漏洞挖掘就像拿着建筑图纸找弱点——App 源码已经在你桌上了，核心能力是"系统性覆盖所有维度"，不能东试西试漏掉高价值方向。

我们用一个 targetType 参数切换这两套打法。同一个 targetType 一路贯穿系统提示、知识库、决策执行器、验证器： 覆盖率闭合：系统要求覆盖率不到 80% 就不能宣布结束。这防止了"找到一个 nodeIntegration: true 就交差"——配置缺陷只是现象，你必须证明它能被利用。

> 小白版：同一套大脑，两个检查清单。检查网站用"密码/注入/越权"清单，检查桌面 App 用"IPC/配置/密钥/提权"清单。每个清单都要过完才能交差。

九、AST 污点分析：不是 grep exec(，是真正的代码分析

大部分"代码审计"工具是正则匹配——在源码里 grep exec(，然后报一个"可能的命令注入"。问题是：exec("echo hello") 和 exec(args.userInput) 在正则看来长一样。我们的污点分析用 TypeScript Compiler API 做真正的 AST 级追踪：

```
function containsTainted(node: ts.Node, tainted: Set<string>): boolean {
  let hit = false
  const visit = (n: ts.Node): void => {
    if (hit) return
    if (ts.isIdentifier(n) && tainted.has(n.text)) {
      hit = true
      // 这个参数引用了被污染的变量
      return
    }
    ts.forEachChild(n, visit)
  }
  visit(node)
  return hit
}
```

工作流程：

> 小白版：就像追踪一条污染的河流。从 IPC 接口（入口）开始，一路顺着代码走，看它最终流到了哪里。如果流到了 exec（执行命令），那就是命令注入。正则只能看"这个池塘边有没有 exec"，我们能看"这个池塘里的水有没有流到 exec"。
>
> 内行版：sinkKind(call) 函数匹配 4 类 sink：child_process.* → critical（命令注入）、shell.openExternal → high（URL 注入）、fs.*File* → high（路径遍历）、.node FFI → high（内存不安全）。不是模式匹配，是数据流证明。

十、Host Worker：从"现象"到"漏洞"的最后一步

静态审计发现 nodeIntegration: true + IPC handler 无 sender 校验 + AST 污点确认数据流——这些都是现象。系统会把它们降级为 phenomenon，不计入漏洞统计。为什么？因为"配置不安全"不等于"能被利用"。就像你看到一栋楼的门没锁，不等于有人真的进去了——你得证明有人能进去。Host Worker 用三个工具补上这最后一步：

第一步：启动 App （`electron_launch`）

- 用 `Bun.spawn` 启动目标 App，加上 `--remote-debugging-port=9222`（开启 Chrome DevTools Protocol）
- CDP 绑定 `127.0.0.1` only，不会暴露到外网
- 等待 CDP 目标出现，返回 session_id

第二步：注入 payload （`cdp_eval`）

- 通过 CDP 连接到 App 的渲染进程
- 执行 JavaScript：`ipcRenderer.send('pushUpdate', {productName: 'app"; touch /tmp/PWNED; #'})`
- 返回执行结果（成功/异常）

第三步：观察副作用 （`side_effect_observe`）

- 轮询文件系统：`/tmp/PWNED` 存在吗？内容包含预期字符串吗？mtime 是新的吗？
- 最长等 5 秒，每 200ms 检查一次
- 返回 hit: true/false

工具产物带幂等键落盘，重启后从历史恢复学习状态。Agent 不是从零开始，是越跑越准。

> 小白版：就像一个新手安全研究员，第一次扫描不知道该先测什么，但测了几次之后，他知道"这个目标的登录接口最容易出问题"，下次就优先测那里。
>
> 内行版：isHit(output, isError) 从工具输出中提取命中信号（verdict=CONFIRMED / finding_count > 0 / confirmed/vulnerable 关键词），累积统计 per-tool per-payload 命中率。

> suggestArgs 从命中历史中提取高频参数填入空缺。幂等键 = SHA-256(sessionId + intentId + toolName + args)，相同调用不重复存储。

十一、在线策略学习：Agent 越跑越准

传统扫描器的规则是写死的——不管扫了多少次目标，下次还是用同样的策略。我们的 Agent 是边跑边学的。每轮工具调用后，自动记录：用了什么工具、传了什么参数、结果有没有命中。然后三个应用：

> 小白版：就像一个新手安全研究员，第一次扫描不知道该先测什么，但测了几次之后，他知道"这个目标的登录接口最容易出问题"，下次就优先测那里。

> 内行版：isHit(output, isError) 从工具输出中提取命中信号（verdict=CONFIRMED / finding_count > 0 / confirmed/vulnerable 关键词），累积统计 per-tool per-payload 命中率。suggestArgs 从命中历史中提取高频参数填入空缺。幂等键 = SHA-256(sessionId + intentId + toolName + args)，相同调用不重复存储。

十二、兜底机制：模型不听话怎么办

toolChoice: "required" 不是所有模型都遵守。有些模型会在你强制它调用工具时，仍然只输出文本。我们的解决方案：确定性执行器兜底。如果模型光说不做，系统从规划层的决策中合成工具调用，强制前进。如果模型调了错误的工具（比如规划层决定做协议分析，但模型调了 RCE 测试），系统用确定性映射替换。

> 小白版：就像导航软件。你说了"去机场"，但司机偏要往反方向开。导航会说"前方掉头"，如果司机还不听，导航直接接管方向盘。

> 内行版：reconcileToolCallsWithDecision(calls, decision, bn, target) 检查模型调用的工具是否和决策层选的工具在同一族（TOOL_FAMILY 映射）。如果不在同一族且是 risky mismatch 工具（framework_rce / sqli_verify / xss_verify 等），替换为决策层的确定性选择。

十三、崩溃恢复：Worker 挂了不丢进度

Worker 的信念图每轮都序列化为快照，写入黑板。Worker 崩溃后，新启动的 Worker 从黑板上读取最新快照，无缝继续。

- **Worker 挂了** ：最新信念在黑板上，新 Worker 接手不丢进度
- **并发 Worker** ：A 这轮确认了一个漏洞，B 下轮立刻能看到
- **人类可审计** ：黑板数据是结构化的 JSON，不是自由文本

> 小白版：就像团队共享的文档。你写了半段崩了，重新打开文档，你写的内容还在，别人也能看到。

结语

两个核心认知：

> 确定性规则 > Prompt 祈祷。 你不能指望 AI 自己判断"这个发现是不是真漏洞"，就像你不能指望一个实习生自己判断自己的代码有没有 bug。你需要的是 7 道确定性关卡——纯代码，零 API 成本——让假漏洞在编译期就被拦住。LLM 二次验证是补充，不是主力。

> 现象 ≠ 漏洞。 nodeIntegration: true 是配置现象，不是漏洞。漏洞是完整的闭环：静态审计发现配置 → AST 污点确认数据流 → CDP 注入 payload → 文件系统观察副作用 → 确认。从"这扇门没锁"到"我用铁丝把锁打开了，录了视频"——中间差的不是技术，是认知架构。